

PROJECT SCOPE	TARGET STACK	DOCUMENT VERSION
AI Preview Generation	Flask / Gemini API / Cloudinary	v1.0 (Strict Execution)

This execution guide translates the global 5-Day Delivery Workflow and Git Monorepo rules directly into an operational step-by-step framework for the **Module 5 Engineering Team**. Every team member must follow this calendar, naming architecture, and code contract precisely to eliminate Day 4 integration failures.

1. Team Branch Assignments & Task Taxonomy

To preserve parallel development speed without mutual blocks, work is distributed across four highly specialized feature branches. All task branches branch off from the unified Module 5 parent branch.

Team Member	Assigned Git Feature Branch	Core Scope & Implementation Rules
NAVEEN KUMAR D (Core Engine)	feat-m5-t1-core-prompt	Orchestration engine (ai_service.py), prompt assembly from category templates (prompt_builder_service.py), Gemini API client wrapper, and injection threat sanitization.
SAWTHI M (Data Layer)	feat-m5-t2-validation-cache	Validation bridge to Module 4, deterministic configuration hashing algorithm, global caching logic (ai_cache_service.py), and custom_text bypass guards.
GOKUL B (Security Guard)	feat-m5-t3-rate-limiting	Two-tier security logic (ai_rate_limit_service.py), anonymous tracking using guest session_id UUIDs, and daily authenticated caps. Maximize hit exception policies.
MOHAMMED (Infrastructure)	feat-m5-t4-routes-cloudinary	Flask API routing layer (ai_routes.py / admin_ai_routes.py), downstream Cloudinary asset pipe mapping to galxy/ai-previews/, and model schemas.

2. Micro-Timeline & Daily Execution Cadence

Day 1: Base Initialization & Shared Assets (Isolation Build)

- **AM Kickoff:** Initialize local environments. Pull the unified dev branch from origin and establish your feature branch locally.
- **Shared Utility Ingestion:** Immediately pull down Module 1's auth decorators and Module 10's Cloudinary base helper as soon as they are pushed to dev. Do not write localized duplicates.
- **Seed Ingestion:** Execute `python seed_categories.py` inside your backend folder to provide a persistent database fixture for testing custom properties.

Day 2: Feature Maturity & First Synchronization Loop

- **AM Synchronization (Mandatory):** Sync with the project-wide development stream by running the exact merge checklist below before writing code.
- **Parallel Coding:** Build out your respective services according to strict functional isolation boundaries. Focus heavily on individual test coverage using mock interfaces.

Day 3: Internal Integration & Feature Freeze

- **AM Synchronization:** Perform the final scheduled merge from `origin/dev` to catch external schema drifts early.
- **PM Self-Integration Check:** Merge all four team member branches into the parent `module-5-ai-preview` local branch. Test full end-to-end telemetry (Route → Rate Limit → Cache Check → Prompt Gen → Gemini Mock → Cloudinary Asset Pipeline).
- **21:00 Deadline:** Freeze features. Push finalized parent branch to origin and flag readiness to the Project Head.

Day 4: Global Integration Gate

- **Standby Protocol:** All Module 5 developers must remain active to support the Project Head during sequential integration mapping. Module 5 merges 6th in order. Fix semantic bugs instantly if escalated.

3. Mandatory Command Reference & Commit Rules

Commit messages must follow the exact syntax specification to allow automated chronological sorting:
`[module-5] type: short description.`

```
# Day 1: Initialize Task Branch Off the Parent Module Branch
git checkout -b feat-m5-t1-core-prompt origin/module-5-ai-preview

# Mandatory Day 2 & Day 3 AM Sync Process
git checkout module-5-ai-preview
git fetch origin
git merge origin/dev
git checkout feat-m5-t1-core-prompt
git merge module-5-ai-preview

# Acceptable Conventional Commit Format Standard
git commit -m "[module-5] feat: implement dynamic placeholder parser for category prompts"
git commit -m "[module-5] fix: enforce strict sanitization on custom_text inputs"
git commit -m "[module-5] style: apply dark neon design system toast triggers"
```

CRITICAL INTEGRATION GUARDRAILS:

Module 5 does NOT author category attribute schemas (owned by Module 2), does NOT validate configurations manually (delegates entirely to Module 4), and does NOT calculate financial pricing metrics (pulls independent estimates via Module 4). Guard these structural contracts carefully.